AI Instruction Sheet

Research Tool for Quantifying Computation Reduction in Smoothed Slices

Comparative implementation for Plaza1 / Plaza2 and synthetic SLAM case studies

One-sentence mission

Build an instrumented MATLAB/App-Designer research tool that can test whether Smoothed Slices reduces nested-sampling work by replacing repeated sample trees with finite conditional-smoothing surfaces, while preserving posterior recovery quality against Slices and NF-iSAM baselines.

1. Source-derived basis and scope

This sheet is an implementation and research guide, not an author-specified algorithm. The original Slices paper specifies variable elimination, the local joint density, the Monte Carlo slice estimator, Lemma-1 sample generation, the conditional estimator, backward reconstruction, and MMD early stopping. Smoothed Slices is the proposed extension: it interprets recursive nested sampling as a tower of conditional expectations and tests whether finite conditional-smoothing surfaces are cheaper or more accurate.

Source item	What the AI must extract/use	Do not overclaim
Slices paper	Forward pass, Eq. (12)/(13)-style slice estimator, Lemma 1 sampleability, Eq. (16)-region nested sampling, conditional estimator, backward pass, MMD early stopping.	Do not claim Smoothed Slices appears in the paper. It does not.
NF-iSAM paper	Bayes-tree clique conditionals, flow training/extraction pipeline, Algorithms 1-3, RQ spline settings, Plaza likelihood models, incremental changed-subtree logic.	Do not replace NF-iSAM with Smoothed Slices. It is a baseline.
Recursive Conditional-Smoothing Slices note	Definitions of conditional-smoothing surfaces R_r, finite-cardinality matrices, sparse active successor sets, tower-property variance argument, surface-guided proposals.	Treat as proposed research formulation that must be validated empirically.

2. Open research question to answer

Primary question: When does Smoothed Slices reduce computations, and by how much, relative to the original Slices nested sampling implementation?

The AI must not assume the answer. The tool must measure it under controlled case studies and produce plots that reveal whether the conditional-smoothing surfaces are genuinely simpler than the nested sample tree.

Hypothesis to test

If the conditional-smoothing surfaces R_r are lower-rank, sparse, smooth, or well-approximated by small active successor sets, then the working cost can move from nested-tree-like growth toward sparse-surface recursion cost.

Use the following conceptual comparison throughout the tool:

```
Paper-like nested work:       Cost_nested(H) ~ N^H * |S|
Smoothed-surface work:        Cost_RCS(H)    ~ sum_{r=0}^{H-1} |X_r| * |N_r| * |S|
Potential gain condition:     sum_r |X_r||N_r||S|  <<  N^H |S|
Important caveat:             This only holds if |N_r| << |X_{r+1}| and R_r is compact.
```

3. Cardinality notation the tool must display

All cardinality quantities must be displayed in the MATLAB app, saved to diagnostics, and used in plot titles or legends where relevant. The app UI should write cardinalities in blue, using vertical bars: **|X_r|, |S|, |N_r|, |Phi_r|, |A_K|**.

Symbol	Meaning in implementation	Where to log it
|X_r| = B_r	Number of finite support points for path variable xi_r.	Per increment, per affected chain, per method mode.
|S| = R_s	Number of separator support/evaluation points used to represent the new factor.	Every factor construction and marginal plot.
|N_r(a)| = K_r(a)	Active successor count used for sparse conditional-smoothing recursion.	Per surface row; store mean, median, max, histogram.
|Phi_r| = K_phi	Basis size if using separator-basis compression.	Surface approximation mode and compression reports.
N	Nominal samples per Slices elimination step.	Baseline Slices and NF-iSAM sample budgets.
H	Depth of the local recursive sampling path.	Synthetic chain experiments and real affected subtrees.
rank_eps(R_r)	Numerical rank of surface R_r under tolerance eps.	Surface complexity diagnostics.
nnz_eps(R_r)	Number of entries above threshold eps.	Sparsity and compressibility diagnostics.

4. Required working-directory hierarchy

The AI/developer must build the tool under a clear project hierarchy. The goal is reproducibility: every instruction, paper, dataset, run, diagnostic, and exported figure has a fixed place.

```
SmoothedSlicesResearchTool/
  Instructions/
    01_App_Spec_Smoothed_Slices.docx
    02_Slices_Exact_Implementation.docx
    03_NF_iSAM_Exact_Implementation.docx
    04_Smoothed_Slices_Exact_Implementation.docx
    05_Plaza_Case_Study_Showcase_Manual.docx
    06_This_Research_Instructions_Sheet.docx
  Literature/
    A_Slices_Perspective_2024.pdf
    NF_iSAM_2023.pdf
    iSAM2_reference.pdf
    Optional_MMD_or_kernel_references.pdf
  Data/
    Plaza1/raw/
    Plaza1/processed/
    Plaza2/raw/
    Plaza2/processed/
    SyntheticChains/
    FourDoors/
  src/
    app/SmoothedSlicesComparatorApp.mlapp
    core/factors/
    core/graphs/
    methods/slices/
    methods/nfisam/
    methods/smoothed_slices/
    diagnostics/
    plotting/
    utils/
```

```
runs/
  yyyy-mm-dd_HHMMSS_case_method_seed/
    config.json
    metrics.csv
    diagnostics.mat
    figures/
    snapshots/
exports/
  paper_style_figures/
  app_diagnostics/
  comparison_reports/
```

5. Core implementation modules

Module	Required responsibility	Critical outputs
CaseStudyLoader	Load Plaza1, Plaza2, FourDoors, and synthetic chains. Standardize variables and units. Build factor graph incrementally.	FactorGraph object, ground truth, measurement stream, metadata.
FactorLibrary	Evaluate prior, odometry, range, ambiguous data-association likelihoods, loop-closure factors, and generated separator factors.	Function handles with consistent call signatures and vectorized evaluation.
SlicesEngine	Implement the authors' Slices method: forward elimination, sample generation through Lemma-1 path, Eq. (13)-style new factors, conditionals, backward slices, MMD early stop.	Posterior samples, marginals, f_new objects, elimination snapshots.
NFiSAMEngine	Implement or wrap the NF-iSAM baseline as closely as possible: Bayes tree, clique density training, separator and conditional samplers, changed-subtree update.	Posterior samples, clique diagnostics, training time/loss, flow dimensions.
SmoothedSlicesEngine	Implement conditional-smoothing surfaces R_r, sparse recursion, surface cache, surface-guided proposal, and fallback to original Slices when surfaces fail.	R_r surfaces, active sets, surface errors, new factors, posterior samples.
ComplexityProfiler	Count every relevant operation, not just wall time.	Factor evaluations, sample draws, surface updates, active successors, memory, runtime, GPU/CPU usage.
SurfaceAnalyzer	Quantify whether R_r is compact.	Rank curves, singular values, sparsity, approximation error, K-vs-error curves.
PlotManager	Generate figures in paper style and research-style diagnostics.	PNG/PDF/MATFIG exports, snapshots for app slider.

6. Instrumentation: what must be counted

The AI must instrument the code before optimizing anything. The open question is quantitative, so every method must expose identical counters when possible.

Counter	Meaning	Why it matters
numFactorEvaluations	Total calls to likelihood/factor functions.	Most direct proxy for algorithmic work in Slices-like methods.
numSampleDraws	Total samples drawn from priors, sampling factors, flows, or proposals.	Shows whether Smoothed Slices really reduces nested sampling.
numNestedBranches	Number of realized sample branches in the original nested view.	Measures growth of the Lemma-1 sampling tree.
numSurfaceEntries	Total entries stored/evaluated in R_r matrices.	Measures surface-cardinality cost.
numActiveSuccessors	Total entries actually used in sparse recursion.	Main variable behind practical savings.
surfaceUpdateTime	Time to build/update R_r.	Checks if smoothing overhead is worth it.
posteriorSamplingTime	Time to produce posterior samples after factors/flows/surfaces are ready.	Important for online use.
methodTotalTime	End-to-end time per incremental step.	Primary runtime metric.
peakMemoryMB	Approximate peak memory.	Surfaces may trade time for storage.
MMD/RMSE/NLL/modeRecovery	Quality metrics.	Savings are meaningful only if posterior quality remains acceptable.

```
Diagnostics struct contract:
Diagnostics.methodName
Diagnostics.caseName
Diagnostics.stepIndex
Diagnostics.seed
Diagnostics.wallTime.total
Diagnostics.wallTime.forward
Diagnostics.wallTime.backward
Diagnostics.wallTime.surfaceUpdate
Diagnostics.counters.factorEvaluations
Diagnostics.counters.sampleDraws
Diagnostics.counters.nestedBranches
Diagnostics.cardinality.Xr, Diagnostics.cardinality.S, Diagnostics.cardinality.Nr
Diagnostics.surface.rank, Diagnostics.surface.singularValues, Diagnostics.surface.sparsity
Diagnostics.quality.RMSE, Diagnostics.quality.MMD, Diagnostics.quality.NLL
Diagnostics.cache.hitRate, Diagnostics.cache.updatedSurfaceCount
Diagnostics.figurePaths
```

7. Smoothed Slices algorithmic experiment modes

Implement Smoothed Slices in several modes. The research question is not answered by one version; the tool must compare variants.

Mode	Description	Purpose
RCS-Finite	Store R_r directly on finite supports and update with sparse active successor sets.	Closest to Slices; easiest to debug.
RCS-Basis	Approximate separator dependence using a small basis |Phi_r|.	Tests whether surfaces are compressible.
RCS-Regression	Learn R_r as a conditional expectation surface using kernel/local polynomial/small NN regression.	Tests whether smoothness can reduce storage and sampling.
RCS-Proposal	Use R_{r+1} to guide sampling proposals but keep importance correction/fallback.	Tests whether surfaces improve sampling quality even when not replacing inner loops.
RCS-Hybrid	Use surfaces where compact; fall back to original Slices where not compact.	Most realistic online mode.

```
Smoothed Slices sparse recursion:
for r = H-1:-1:0
    for a = 1:|X_r|
        successors = activeSuccessors(r,a);   % |N_r(a)| = K_r(a)
        for rho = 1:|S|
            R_r(a,rho) = sum_b P_r(a,b) * A_r(a,b,rho) * R_{r+1}(b,rho)
        end
    end
end

where A_r(a,b,rho) contains factor contributions toward separator point s^(rho).
```

8. Experiments the AI must implement

Experiment	Independent variables	Outputs / plots
E1: Synthetic depth sweep	Depth H, samples N, support |X_r|, active K, separator |S|.	Measured work vs H, log-scale; compare N^H vs measured RCS cost.
E2: Synthetic surface complexity	Noise, nonlinearity, multimodality, overlap width.	Rank decay of R_r, sparsity, active K needed for fixed error.
E3: Four Doors mode recovery	Sample budget, ambiguous observations, number of doors/modes.	Mode recovery probability, MMD, RMSE, runtime, branch count.
E4: Plaza2 main reproduction	150 samples Slices-style, NF-iSAM recommended samples, several RCS cardinalities.	Landmark marginal plots over time, RMSE/runtime per step, MMD if available.
E5: Plaza1 scalability	Incremental time horizon, ambiguity fraction, surface mode.	Runtime vs dimension/step, accumulated work, final trajectory/landmarks.

E6: Ablation	Disable caching, disable active successors, disable proposal guidance, disable basis compression.	Identify exactly which mechanism gives the saving.

9. Plaza1 / Plaza2 integration instructions

For Plaza datasets, the tool must follow the same experimental spirit as the papers while making all instrumentation visible. The papers use range-only SLAM with odometry and four unknown landmarks; Plaza range errors may require calibration/bias handling before applying Gaussian noise assumptions. The app must allow data association ambiguity to be toggled and controlled by percentage when reproducing NF-iSAM-style experiments.

Requirement	Implementation detail
Dataset selection	Dropdown: Plaza1, Plaza2, SyntheticChain, FourDoors.
Time window	Controls: start step, end step, step stride, live incremental playback.
Measurement models	Pose odometry factor; range factor with known association; ambiguous range factor as equal-weight mixture over possible landmarks.
Variable ordering	Use a fair ordering across methods. Provide default: poses along trajectory first, landmarks last. Log every ordering used.
Ground truth	Load if available. Use for RMSE and visualization only, never for inference.
Calibration	Allow optional least-squares range-bias calibration for Plaza-style analysis. Save calibrated and raw runs separately.
Seeds	Run multiple seeds for stochastic methods. Store seed list and confidence bands.
Budgets	Expose sample budgets and cardinalities in UI: N, |X_r|, |S|, |N_r|, |Phi_r|.

10. Required plots and diagnostics

The tool must produce both paper-style plots and new research plots that directly answer the open question.

Plot group	Exact figures to generate	Purpose
Posterior recovery	Joint samples; marginal KDEs for poses/landmarks; credible ellipses/contours; landmark L2 over key Plaza2 steps.	Show whether posterior shape is recovered.
Performance	Runtime per step; accumulated runtime; RMSE per step; marginal MMD and joint MMD vs step.	Replicate paper-style method comparison.
Computation reduction	Factor evaluations vs step; sample draws vs step; nestedBranches vs step; surfaceEntries vs step; activeSuccessors vs step.	Directly quantify saved calculations.
Scaling	Log work vs recursion depth H; work vs sample budget N; work vs separator cardinality |S|.	Estimate empirical complexity exponent.
Surface complexity	Singular-value decay of R_r; numerical rank vs step; sparsity heatmap; active-K vs error curve.	Test if R_r is compact enough to justify the method.
Bias-variance	Across-seed variance of f_new estimates; smoother residual; MMD vs cost Pareto curves.	Determine whether saved inner sampling noise is replaced by harmful surface bias.
Process documentation	Slider snapshots of variable elimination, Slices nested sampling path, Smoothed surface update, NF-iSAM clique update.	Help reader understand algorithm evolution.

Most important plot

Plot posterior-quality error versus measured factor-evaluation count, not only versus wall time. If Smoothed Slices gives lower MMD for the same number of factor evaluations, or the same MMD with fewer factor evaluations, the method is genuinely promising.

11. How to decide whether the research idea works

Claim	Evidence required	Failure signal
RCS reduces computations	Measured factor evaluations, sample draws, and wall time are lower than Slices at matched MMD/RMSE.	Surface-update overhead exceeds saved nested sampling.
RCS preserves posterior recovery	MMD, marginal plots, and mode recovery match or improve Slices at equal compute.	Surface approximation smooths away modes or biases marginals.
R_r surfaces are compact	Fast singular-value decay, low rank_eps, small active K, stable basis coefficients.	Full rank/no sparsity; K must approach |X_{r+1}|.
Caching helps online inference	High cache hit-rate and smaller update region on Plaza increments.	Most surfaces must be rebuilt from scratch each step.
Proposal guidance helps	Higher ESS and lower estimator variance with RCS-guided proposal.	ESS does not improve or support is lost without defensive fallback.

12. Developer task list

Phase 1 - Baseline reproducibility

- Implement/import Plaza1 and Plaza2 loaders.
- Implement identical factor library for all methods.
- Run original Slices baseline and confirm f_new / marginal plots on small examples.
- Run NF-iSAM baseline or compatible wrapper; log training/sample settings.
- Create a single MethodRunner interface for all methods.

Phase 2 - Instrumentation

- Wrap every factor function to increment factor-evaluation counters.
- Wrap every sampler to increment sample-draw counters.
- Implement nested-branch counter for Slices.
- Implement surface-entry and active-successor counters for Smoothed Slices.
- Save Diagnostics struct to .mat, .json, and .csv after each run.

Phase 3 - Smoothed Slices implementation

- Implement finite support spaces X_r and separator support S.
- Implement R_r matrix update and active successor sets N_r(a).
- Implement surface cache between increments.
- Implement basis and regression versions only after finite mode is verified.
- Implement fallback: if surface residual/rank/K is too high, run original Slices for that block.

Phase 4 - Plots and research dashboard

- Add App panels for posterior recovery, performance, complexity, surface analysis, and process slider.
- Add export-all button that saves figures, diagnostics, configs, and snapshots.
- Generate paper-style figures plus new complexity figures for the open question.
- Add LaTeX interpreter for all titles, labels, legends, and UI equation text where MATLAB supports it.

13. App integration and process sliders

The app should not only run methods; it should teach the difference between them. Add method-specific process panels with sliders:

Slider panel	What each slider index shows
Slices elimination slider	Current eliminated variable theta_j, removed factors F_{j-1}(theta_j), separator S_j, selected sampling factor f', generated samples, and f_new estimate.
Smoothed Slices surface slider	Path level r, support points X_r, active successors N_r(a), surface R_r heatmap, rank/sparsity metrics, and resulting f_new.
NF-iSAM clique slider	Current Bayes-tree clique C, frontal variables F_C, separator S_C, training-sample generation, flow loss, extracted samplers.
Incremental step slider	Graph growth, affected subtree, cached objects reused, new factors inserted, backward early-stop point.

14. Minimal pseudocode for the research tool

```
for caseName in {SyntheticChain, FourDoors, Plaza1, Plaza2}
  data = CaseStudyLoader(caseName, config)
  for seed in seeds
    for method in {NF_iSAM, Slices, SmoothedSlices}
      reset Diagnostics
      for k = 1:numIncrements
        graph_k = data.buildGraph(k)
        result_k = MethodRunner(method).update(graph_k, cache, config)
        metrics_k = evaluatePosterior(result_k.samples, groundTruth, referenceSamples)
        Diagnostics.log(k, result_k, metrics_k)
        cache = result_k.cache
      end
      PlotManager.exportRun(Diagnostics)
    end
  end
  PlotManager.compareMethods(caseName)
end
```

15. Reporting rules for the AI

- Always report whether a result is source-derived, implemented baseline behavior, or proposed Smoothed Slices behavior.
- Never claim complexity reduction from theory alone. Report measured counters and confidence intervals across seeds.
- Always compare at matched posterior quality: same MMD/RMSE/mode recovery threshold, then compare computations.
- If Smoothed Slices is faster but loses modes, report failure honestly.
- If Smoothed Slices is not faster, inspect why: high surface rank, too many active successors, cache invalidation, or surface regression overhead.
- Use figures to answer: Are the surfaces compact? Does compactness translate into fewer factor evaluations? Does that preserve posterior shape?

16. Acceptance checklist

Checkpoint	Pass condition
Baseline fidelity	NF-iSAM, Slices, and Smoothed Slices run on the same factor graph and measurement models.
Instrumentation	All methods save factor evals, sample draws, memory, runtime, and quality metrics.
Complexity plots	There is at least one plot directly comparing measured nested work and surface-recursion work.
Surface plots	Every Smoothed Slices run exports rank/sparsity/active-successor

	diagnostics.
Plaza demonstration	Plaza1 and Plaza2 produce trajectory, landmark marginal, runtime, RMSE, and method comparison figures.
Research conclusion	The final report states whether the hypothesis is supported, rejected, or unresolved for each case study.

References to keep in the project Literature folder

1. M. Shienman, O. Levy-Or, M. Kaess, and V. Indelman, “A Slices Perspective for Incremental Nonparametric Inference in High Dimensional State Spaces,” IROS 2024.
2. Q. Huang, C. Pu, K. Khosoussi, D. M. Rosen, D. Fourie, J. P. How, and J. J. Leonard, “Incremental Non-Gaussian Inference for SLAM Using Normalizing Flows,” IEEE Transactions on Robotics, 2023.
3. Recursive Conditional-Smoothing Slices: A tower-property formulation for reducing nested sampling in the Slices perspective, internal formulation note.
4. Optional supporting references: iSAM2 / Bayes tree, MMD two-sample test, conditional mean embeddings, Rao-Blackwellization, sparse Gaussian process or kernel regression if used for surface learning.